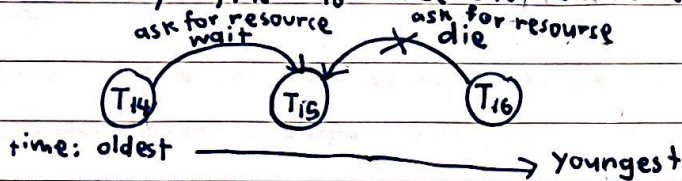


1) Explain the meaning of the following terms:

①.1 Wait-Die Deadlock prevention scheme

The wait-die deadlock prevention scheme is a nonpreemptive technique. When transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if it has a timestamp smaller than that of T_j (i.e. T_i is older than T_j). Otherwise, T_i is rolled back (dies). For example, suppose that transactions T_{14} , T_{15} , T_{16} have timestamps 5, 10, and 15, respectively. If T_{14} request a data item held by T_{15} , then T_{14} will wait. If T_{16} requests a data item held by T_{15} , then T_{16} will be rolled back (dies).

e.g.



①.2 Intention lock modes

▷ In addition to Shared (S) and Exclusive (X) lock modes, there are 3 additional lock modes with multiple granularity:

◁ • Intention-Shared (IS)

↳ indicates explicit locking at a lower level of the tree but only with shared locks.

• Intention-exclusive (IX)

↳ indicates explicit locking at a lower level with exclusive or shared locks

• Shared and intention exclusive (SIX)

↳ the subtree rooted by that node is locked explicitly in shared mode and explicit locking is being done at a lower level with exclusive-mode locks.

▷ Intention locks allow a higher level node to be locked in S or X mode without having to check all descendents nodes

1.3 write timestamp(Q)

↳ denotes the largest timestamp (youngest transaction) of any transaction that executed write(Q) successfully

1.4 write ahead logging (WAL)

↳ Before a block of data in main memory data can be output to the database (in non-volatile storage), all log records pertaining to data in that block must have been output to stable storage

↳ Namely, log record must be output to the logfiles before the corresponding modified database buffer is output to DBSpace

1.5 checkpoint

In case of system crashes, we can avoid redoing all changes from the beginning of the entire log by using checkpoint

② Slow Changing Dimensions (SCDs)

↳ Dimension that stores and manages both current and historical data over time in a data warehouse which can change slowly but unpredictably

②.1 Problem occurs when the existing rows in dimension tables need to be updated. Although existing rows in dimension tables need to be updated occasionally, this happens infrequently.

For example, a database may contain a fact table that stores sales records. This fact table would be linked to dimensions by means of foreign keys. One of these may contain data about the company's salespeople e.g. regional offices in which they work. However, the salespeople are sometimes transferred from one regional office to another. For historical sales reporting purposes it may be necessary to keep record of fact that a particular sales person had been assigned to a particular regional office at an earlier date, whereas that sales person is now assigned to a different regional office. ∴ incorrect information is created and maybe the data warehouse can be outdated

②.2 Solutions

① overwrite the old attribute values

↳ existing data is lost as it is not stored anywhere else

Name	Region
Mine	USA UK

② create another Dimension record

↳ retains the full history of values

↳ when the value of a chosen attribute changes, the current record is closed

↳ a new record is created with the changed data values and this record becomes the current record.

↳ Each record contains the effective time, and expiration time to identify the time period between which record was active

Name	Region	ValidFrom	ValidTo	Newest	version
Mine	USA	01/05/21	01/08/21	No	1
Mine	UK	02/08/21	31/12/99	Yes	2

③ Creating a current value field .

- stores 2 versions of values for certain selected level attributes
- Each record stores the previous value and the current value of the selected attribute .
- When the value of any of the selected attribute changes, the current value is stored as the old value and new value becomes the current value

Name	OldRegion	NewRegion
Mine	USA	UK

②.3 With surrogate keys, the old fact table rows continue to refer to the original dimension table rows, while new fact table rows refer to new dimension table rows. Additionally, it enables the capture of correct information in the data warehouse

▷ The first time a fact table row refers to a row in a dimension table can be determined, and this time can serve as conservative bound on when the dimension table row came to existence. The exact time when a change happens in a dimension can be captured by inserting a special row in the fact table. This row refers to the new dimension row and to the row in the time dimension table that represents the time when changes happen .

③

Distributed Database Architecture should comprise

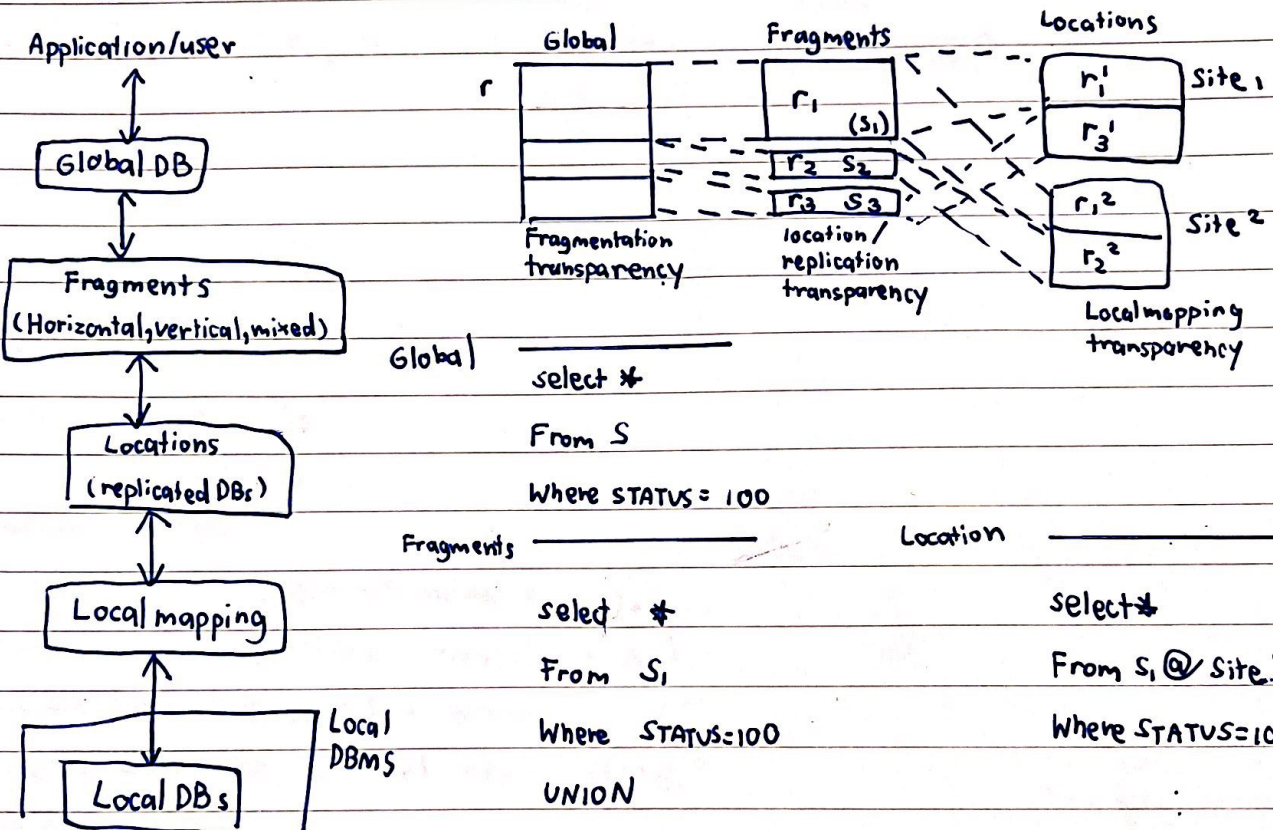
- Autonomous DBMS, Database
- Interconnected via a computer network
- Local applications: access only local databases
- Global applications: access databases from other sites

distributed data processing

ex: open bank at Ladkrabang

↳ withdraw money at Chiangmai

↳ Data will be processed at Ladkrabang



6

7

NO. 62011192 Pattenin
DATE / /

③ cont.

EMP

E# ENAME SALARY DEPT TAX

EMP₁

E1 Mine 50,000 5 500

1-10 DEPT

11-20 DEPT

EMP₂

E1 Mine 50,000 15 500

move

to 15

→

select

insert

delete

EMP₁₁

EMP₁₂

EMP₂₁

EMP₂₂

E# ENAME DEPT

E# SALARY TAX

E# ENAME SALARY

E# DEPT TAX

Mixed fragmentation

Begin Tx

select * into A₁, A₂, A₃

From EMP₁₁ @ Site 1

Where E# = "E1"

select * into A₄, A₅

From EMP₁₂ @ Site 2

where E# = "E1"

insert into EMP₂₁ @ Site 3 values(A₁, A₂, A₄)

insert into EMP₂₂ @ Site 3 values(A₁, A₃, A₅)

Delete from EMP₁₁ @ Site 1 where E# = "E1"

Delete from EMP₁₂ @ Site 1 where E# = "E1"

End Tx

4

$$(4.1) \pi_{SNAME} \left(\left(\pi_{S\#, SNAME} \left(\sigma_{CITY = \text{"BANGKOK"}} \right) \right) \bowtie \pi_{S\#} \left(\sigma_{P\# = \text{"P5"}} \right) \right)$$

$$\pi_{SNAME} \left(\sigma_{CITY = \text{"LONDON"} \wedge P\# = \text{"P5"}} \right)$$

$$(4.2) \pi_{SNAME} \left(\left(\pi_{S\#, P\#} (SP) \div \pi_{P\#} (P) \right) \bowtie S \right)$$

select SNAME

From S, SP

where S.S# = SP.S#

AND CITY = "Bangkok"

AND CONTAIN ALL (P#)

⑤ Two phase locking protocols ensures serializability.

▷ This protocol requires that each transaction issue lock & unlock requests in two phases:

1. Growing phase: A transaction may obtain locks, but may not release any lock

2. Shrinking phase: A transaction may release locks, but may not obtain any new locks

▷ This protocol doesnot ensure freedom from deadlocks

▷ Give correct output

▷ can solve lost update and inconsistent analysis problem

Lock-compatibility matrix

	S	X
S	T	F
X	F	F

Lost update

T_A

time

—	—
Fetch R R (acquire S lock on R)	+1 —
—	—
—	+2 Fetch R R (acquire S lock on R)
—	—
—	—
UPDATE R (request x lock on R)	+3
wait	—
wait	+4 UPDATE R (request X lock on R)
wait	—
wait	wait
wait	wait

have to wait bc of lock-compatibility matrix

∴ deadlock occurs at time T₄

solve without deadlock? :

wait mode!

use x lock R at the beginning, so when T_B tries to read R it has to wait & then we will not obtain wrong result

5 cont

Inconsistent analysis

Acc 1	Acc 2	Acc 3
40	50	30
T_A	time	T_B
=		=
Fetch R Acc1(40) (acquire S lock on Acc1)	+1	=
sum = 40		=
Fetch R Acc2(50) (acquire S lock on Acc2)	+2	=
sum = 90	+3	Fetch R Acc3(30) (acquire S lock on Acc3)
=		=
=	+4	UPDATE Acc3 (acquire X lock on Acc3) 30 → 20
=		=
=	+5	Fetch R Acc1(40) (acquire S lock on Acc1)
=		=
=	+6	UPDATE Acc1 (request X lock on Acc1)
=		wait wait wait wait
Fetch R Acc3(20) (request X lock on Acc3)	+7	
wait wait	↓	

∴ Deadlock on +7

solve without deadlock?

Locking entire table, in this case, Acc table

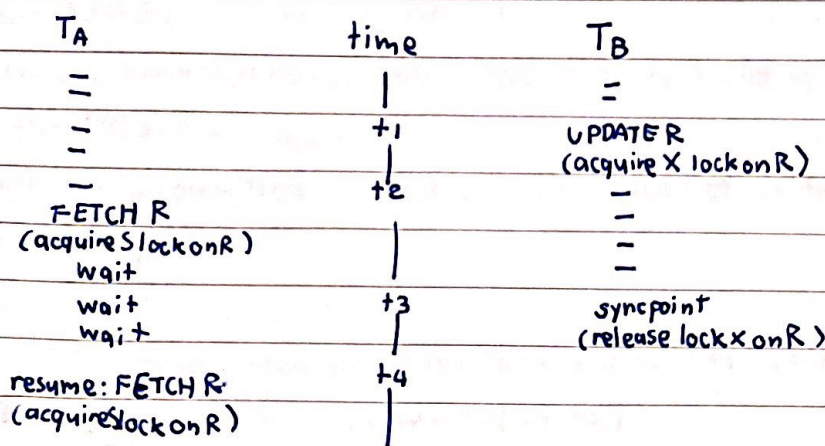
↳ may be overdo, this is where multiple granularity technique comes to fix: size of data to be locked can be adjusted

5) cont.

NO. 62011192 Pattani
DATE / /

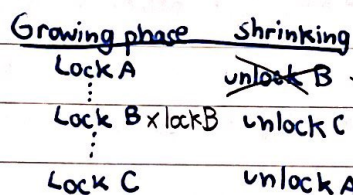
10

uncommitted problem



cascading rollback can be avoided by a modification of strict two-phase locking protocol.

→ 2-phase locking + X lock



← must be released at syncpoints
(commit/Rollback)

unlock at syncpoints

Phantom phenomenon

→ use strict two-phase locking with multiple granularity without deadlock

★ In short, to solve 4 concurrency control problems without deadlock is to use strict 2 phase locking with waitmode: X-lock at the beginning + Multiple granularity

⑥

problem statements (How to recover?)

- ① There are committed transactions whose modified buffer values are not yet output to the DBspace on the disk
- ② There are uncommitted transactions whose modified buffer values are already output to the DBspace

How to solve?

Step 1: The recovery manager of the DBMS will scan the log file from the back to determine which transaction is committed or fail

Step 2: transaction id of committed transactions will be kept in a list called redo list.
————— , , ——— failed ——— , , ——— undo list.

Step 3: for each transaction in undo list, old values are applied to DBspace
(solve problem ②)

Step 4: for each transaction in redo list, committed transactions' new values are applied
(solve problem ①)

Bi; ONLY!!!

How it works

Commit:

DBMS must output all the related data to the DBspace first then declare that transaction commit the discard the old value in that log file

the modified buffer of the transaction must be output completely to the DBspace to declare the transaction commit

Rollback: Normal

Advantages; No A_{ij} full problem

; save storage space, since there is no A_{ij}

: only undo (old value) no redo → fast recovery

disadvantages: takes longer time to commit